Decentralized Tabu Searches in Multi Agent system for Distributed and Flexible Job shop Scheduling Problem

Bilel Marzouki
Ecole Nationale des Sciences de l'Informatique
SOIE-COSMOS Laboratory, ENSI
Université de la Manouba, Tunisia
Email: bilel.marzouki@ensi-uma.tn

Olfa Belkahla Driss
Ecole Supérieure de Commerce de Tunis
SOIE-COSMOS Laboratory, ENSI
Université de la Manouba, Tunisia
Email: olfa.belkahla@isg.rnu.tn

Khaled Ghédira
Institut Supérieur de Gestion
SOIE-COSMOS Laboratory, ENSI
Université de la Manouba, Tunisia
Email: khaled.ghedira@isg.rnu.tn

***Abstract*—Scheduling in production systems consists in assigning operations on a set of available resources in order to achieve defined objectives. The Flexible Job shop Scheduling Problem (FJSP) is one of the scheduling problems where each operation can be processed on different machine and its processing time depends on the used machine. But in the recent years, many companies decide to move towards the decentralization of their factories which allow it to gain advantages towards its customers. In the case of the classic Flexible Job shop Scheduling Problem, we assume that there is a single factory with a set of m machines and n jobs, but in Distributed and Flexible Job shop Scheduling Problem (DFJSP), there is a set of geographically distributed factories in different locations. Each factory contains m machines on which n jobs must be processed. The Distributed scheduling problems and more specifically the DFJSP are much more complicated than standard problems because they involve not only the problem of assigning jobs to machines but also the problem of distribution of jobs in different factories. So, the DFJSP is harder than the FJSP. The DFJSP is classified, as most of scheduling problems, NP-Hard in complexity theory. In this paper, we propose a decentralized model based on tabu search to solve the Distributed and Flexible Job shop Scheduling Problem in order to minimize the maximum completion time (makespan). To evaluate the performance of our model, a set of experiments are performed on benchmark instances well known in the literature.**

***Index Terms*—Manufacturing, Scheduling, Optimization, Multi Agent System, Tabu Search, Distributed and Flexible Job shop Scheduling Problem, Makespan.**

I. Introduction

Scheduling in production systems consists in assigning operations on a set of available resources in order to achieve defined objectives. The Job shop Scheduling Problem (JSP) is considered as one of the most difficult scheduling problems. This problem consists on assigning a set of operations on a set of machines such as each operation must be processed on one machine. The Flexible Job shop Scheduling Problem (FJSP) is an extension of the classical Job shop Scheduling Problem (JSP) where each operation can be processed on different machines and its processing time depends on the used machine. This problem is known to be strongly NP-Hard [1], that's why the most methods of resolution of this problem are decentralized and based on metaheuristics [2-5].

The Distributed and Flexible Job shop Scheduling Problem (DFJSP), that we study in this work, is an extension of the FJSP and there is a set of geographically distributed factories in different locations such that each factory contains m machines on which n jobs must be processed.

The remainder of this paper is organized as follows. In the next section, we present the literature work, in section 3, we present the problem formulation, the tabu search is defined in section 4, we describe our proposed model namely a Distributed Tabu Searches in Multi Agent system for DFJSP (DTSMA) in section 5, we give some experimental results in section 6 and a conclusion and perspectives are given in section 7.

II. Related Works

New researches, such as [6,7] have shown that companies with multiple production centers are more competitive in a globalized economy. With factories located in different geographical areas, companies can benefit from several advantages such as proximity to customers, can respond quickly to market changes, benefit from significant productivity gains from several factors such as the difference in cost of labor or qualification of personnel in certain areas, etc, that's why such companies with multiple production centers have increased in recent years.

The distributed scheduling problems and more specifically the DFJSP are much more complicated than standard problems because they involve not only the problem of assigning jobs to machines but also the problem of distribution of jobs in different factories. So, the DFJSP is harder than the FJSP. The DFJSP is classified, as most of scheduling problems, NP-Hard in complexity theory. In the literature, there are few works that studied the Distributed and Flexible Job shop Scheduling Problem.

This new concept is proposed for the first time by [8] in 2006, Chan et al. have proposed a solution based on genetic algorithms with dominant genes to solve the DFJSP in order to minimize the Makespan. In the approach of the classical algorithm, during crossing, a certain number of genes will

2161-5330/17 $31.00 © 2017 IEEE
DOI 10.1109/AICCSA.2017.133

IEEE computer society

be randomly selected, governed by a predefined crossing rate. It is difficult to ensure that the important part of the chromosome structure can be selected. Sometimes these selected genes cannot be criticized, same to its original chromosome structure. Indeed, there is a lack of a mechanism to measure and identify the genes that are important. To avoid this problem, the idea of dominant genes (DG) is applied. DG are used to identify and save the best genes in the chromosome during evolution. When a gene is designated as DG, it is represented as a gene that will have the ability to increase its fitness in its chromosome structure. They are an important part of the chromosome. At the beginning of evolution, some genes are randomly assigned as DG. Each chromosome can contain more than one DG. During evolution, only the DG are subjected to the crossover phase in each pair of parents, to generate a pair of sons. Each offspring reserves most of the genes of one parent and inherits only the DG (the important structure identification) of another parent. If these inherited DGs increase the strength of the son, they will be designated as the DGs, otherwise they will be designated by normal genes. This mechanism has the advantage of ensuring that the identified important structure can be inherited by the child. In this algorithm, the chromosome coding is modified, each gene is composed of five parameters representing the factory, machine, job, operation and domination (F,M,J,O,D). If the gene is classified as dominant, parameter D is assigned to 1, 0 otherwise. For the crossover function, the authors performed a simple crossover with one-point crossover method and for the mutation, a pair of gene is selected and randomly swapped. To demonstrate the proposed model efficiency, a series of tests are performed and comparisons made with other models and the results show the effectiveness of the proposed model.
[9] proposed a model based on genetic algorithms to solve the distributed and flexible job shop problem in order to minimize the makespan. For the chromosome representation, the authors proposed that the gene number of a chromosome is equal to the sum of the operations of all jobs and each gene corresponds to the operation followed by the number of factories and the number of jobs respectively. To create the initial population, the authors proposed to create it in two phases: the first phase consists of randomly assigning the jobs to the different factories and creating a gene for each operation with its information related to the factory and the job. The second phase consists of combining randomly the genes and this treatment is repeated until the stopping criterion is reached which is the number of individuals. For the selection phase: a growing sort of makespan is performed on individuals and a well-defined probability to choose the chromosomes that will be used for the crrosover phase which is based on the one-point crossover and two-point crossover method. For the mutation, the authors used two methods of mutation: local mutation and global mutation. For the local mutation, it consists of randomly selecting a number of gene pairs in the same chromosome and swapping their positions. For overall mutation, it is applied at the end of each iteration and is to randomly change the assignment of jobs to factories. After the mutation phase, the chromosome is refined by a local search phase. This approach is tested and compared with other existing approaches on different instances of benchmarks in the literature.
[10] proposed a heuristic based on a constructive procedure to solve the distributed and flexible job shop scheduling problem in order to minimize maximum completion time. To evaluate this algorithm, a series of experiments is carried out on instances of benchmark of the literature and the results show the efficiency of this algorithm. [11] proposed a model based on refined genetic algorithms to solve the distributed and flexible job shop scheduling problem in order to minimize the makespan. This model is tested on real world examples and the results show the effectiveness of this model. The initial population is created from the coding function of the operators. The crossover phase is based on the two-point crossover method and the mutation phase is randomly performed.
[7] proposed a hybridization of genetic algorithms to solve the distributed and flexible job shop scheduling problem in order to minimize the makespan. The proposed approach uses the Taguchi method to optimize the parameters of genetic algorithms. For the initial solution, the authors begin by determining the number of jobs and the number of operations and construct the genes from the previous information. For the crossover phase, the two-point crossover, precedence-preserving order-based crossover (POX), and uniform crossover methods are used. The authors divided the mutation operation into two parts: The first part consists in exchanging two random machines and the second part is to change the assignment of operations to the machines. Experiments are performed to evaluate the performance of the approach used on benchmark instances in the literature and the results show the effectiveness of this approach.
[6] proposed an approach based on genetic algorithms to solve the distributed and flexible job shop scheduling problem in order to minimize the makespan. The authors presented a new chromosome presentation and refinement method to improve scheduling performance. Experiments are performed on benchmark instances known from the literature to evaluate the performance of this approach and comparisons are made with other existing approaches and the results show the competitiveness of the proposed approach. We find also works in the literature that have studied the distributed Scheduling Problem such as [12-18].
We notice that the different existing works in literature that have studied the Distributed and Flexible Job shop Scheduling Problem (DFJSP) are centralized and based on metaheuristic methods, specially on genetic algorithms and since the problem is naturally decentralized, we propose in this paper for the first time a decentralized model, using multi agent system based on a tabu search metaheuristic to solve the DFJSP in order to minimize the completion time (makespan). We are interested in this work to solve the problem of scheduling jobs in DFJSP to minimize the maximum completion time using a multi-agent model based

on tabu search.

III. Problem Formulation

A typical distributed scheduling problem generally consists of a number of factories (F=1, 2, . . . , l) and a number of jobs (N=1, 2, . . . , i). They are geographically separated with a travel time (D_{li}). Each factory has a different number of machines (H_l) and can produce various product types with different efficiency. Each job has up to N_i number of operations, and each operation can be performed on more than one suitable machine (but not all) with different processing times (T_{ijhl}). The problem herein is to determine how to allocate these jobs (N) to suitable factories and then determine the production scheduling in each factory in order to minimize the makespan. Similar to classical scheduling problems, it is generally assumed that each machine can only handle one operation at each time. Each operation will be completed before another operation will be loaded. Note that, once a job is allocated to a factory, all of its operations will be processed in that factory. The Distributed and Flexible Job shop Scheduling Problem can be formulated as follows [8]:

φ_{il} Binary integer, defined as 1 if job i is allocated to factory l, otherwise 0.
σ_{ijhl} Binary integer, defined as 1 if operation j of job i is allocated on machine h in factory l, otherwise 0.
δ_{ijkh} Binary integer, defined as 1 if operation j of job i occupied time slot k on machine h, otherwise 0.
F Number of factories.
N Number of jobs.
D_{li} Travel time between factory l and job i.
N_i Number of operations of job i.
H_l Number of machines in factory l.
K Time horizon under consideration.
T_{ijhl} Processing time of operation j of job i on machine h in factory l.
S_{ij} Starting time of operation j of job i.
E_{ij} Ending time of operation j of job i.
C_i Completion time of job i.

In the problem, it is assumed that F, N, N_i, H_l, and T_{ijhl} are given. The decision variables are φ_{il}, σ_{ijhl}, and δ_{ijkh}. With the solution φ_{il}, σ_{ijhl}, and δ_{ijkh} obtained, the value of S_{ij}, E_{ij}, and C_i can be evaluated. The objective is to minimize the makespan of jobs.

$$\text{Objective Z: minimize}(\max\{C_i\}) \quad (1)$$

Completion time (C_i) of job i equals to the summation of the completion time of the last operation (N_i) of job i and the travel time between factory l and job i as defined in equation (2).

$$C_i = E_i N_i + \sum_l \mathrm{D}_{li}\varphi_{il} \quad (2)$$

Ending time (E_{ij}) of operation j of job i equals the summation of its starting time and the processing time on the allocated machine h as defined in equation (3).

$$E_{ij} = S_{ij} + \sum_{hl} \mathrm{T}_{ijhl}\sigma_{ijhl} \quad (3)$$

The problem is subject to the following constraints:
Precedence constraints:

$$S_{ij} \geq E_{i(j-1)} + 1 \ (\forall\ i=1,2,3 \ldots N;\ \forall\ j=1,2,3 \ldots N_i) \quad (4)$$

Operation constraints:

$$\sum_{jhl} \varphi_{il}\sigma_{ijhl} = N_i (\forall\ i=1,2,3 \ldots N) \quad (5)$$

$$\sum_{hl} \sigma_{ijhl} = 1 \ (\forall\ i=1,2,3 \ldots N;\ \forall\ j=1,2,3 \ldots N_i) \quad (6)$$

$$\sum_{h} \delta_{ijkh} \leq 1 \ (\forall\ i=1,2,3 \ldots N;\ \forall\ j=1,2,3 \ldots N_i;\ \forall\ k=1,2,3 \ldots K) \quad (7)$$

Processing operation constraints:

$$E_{ij} - S_{ij} = \sum_{kh} \delta_{ijkh} (\forall\ i=1,2,3 \ldots N;\ \forall\ j=1,2,3 \ldots N_i) \quad (8)$$

$$\sum_{kh} \delta_{ijkh} = \sum_{l} \mathrm{T}_{ijkhl}\sigma_{ijhl} (\forall\ i=1,2,3 \ldots N;\ \forall\ \mathrm{j}=1,2,3 \ldots N_i) \quad (9)$$

Machine capacity constraints:

$$\sum_{ijl} \delta_{ijkh}\varphi_{il} \leq 1 \ (\forall\ k=1,2,3 \ldots K;\ \forall\ h=1,2,3 \ldots H_l \quad (10)$$

Jobs allocation constraints:

$$\sum_{l} \varphi_{il} = 1 (\forall\ i=1,2,3 \ldots N) \quad (11)$$

In the above constraints, constraint (4) defines that the starting time of each operation can only be processed after its precedence operation. Constraint (5) limits each job that has to be completed in the same factory. Constraint (6) limits each operation to be carried out on only one machine throughout the horizon. Constraint (7) limits each operation to be only carried out on one machine at each time unit. Constraint (8) limits the operation to be proceeded until completion. Constraint (9) ensures the processing time of operation j of job i equals the defined required time. Constraint (10) limits each machine to carry out only one operation at each time unit. Constraints (11) limits each job to be allocated to only one factory.
To explain the DFJSP, a sample problem of 4 jobs, 3 machines and 2 factories is shown in Table 1, where the numbers present in factory column present the number of factory and The other numbers show the processing times of the operations on the different machines. The tags "-" mean that the operation cannot be executed on the corresponding machine and we consider the travel times between the factories and the jobs are negligible. Figure 1 shows a Gantt diagram for a scheduling of the proposed example.

TABLE I
A SIMPLE INSTANCE OF THE DFJSP

job	operation	factory number	M1	M2	M3
Job 1	O_{11}	1	1	2	-
	O_{12}	1	2	-	2
	O_{11}	2	-	2	1
	O_{12}	2	1	2	-
Job 2	O_{21}	1	1	3	-
	O_{22}	1	2	2	-
	O_{23}	1	-	1	2
	O_{21}	2	3	-	4
	O_{22}	2	-	2	2
	O_{23}	2	-	1	1
Job 3	O_{31}	1	-	1	2
	O_{32}	1	3	-	3
	O_{33}	1	4	-	2
	O_{31}	2	1	-	4
	O_{32}	2	2	2	-
	O_{33}	2	2	-	6
Job 4	O_{41}	1	1	-	5
	O_{42}	1	3	-	4
	O_{43}	1	2	-	4
	O_{41}	2	4	-	3
	O_{42}	2	-	2	5
	O_{43}	2	2	1	-

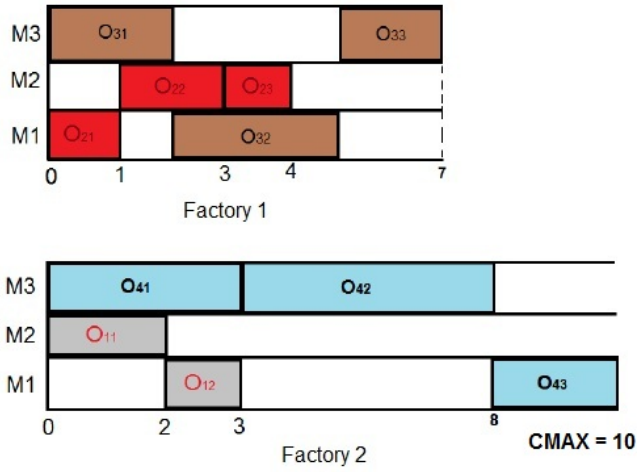


Fig. 1. Gantt diagram of a faisable solution

IV. TABU SEARCH

The Tabu Search (TS) originally proposed by [19] is an iterative meta-heuristic approach based on the principle of the local search and that proved to be successful in solving hard optimization problem. Tabu Search works by starting from an initial solution as current solution and then moves successively among neighboring by applying a move mechanism in the hope of finding an improved solution until some stopping criterion has been satisfied. At each iteration, the process consists in selecting the most appropriate neighborhood which may not be an improving solution. Then the search is repeated starting from the best found permutation as a new current solution, see figure 2. In order to avoid being trapped in suboptimal regions, the Tabu Search uses a memory structure which operates by storing the selected move in a data structure

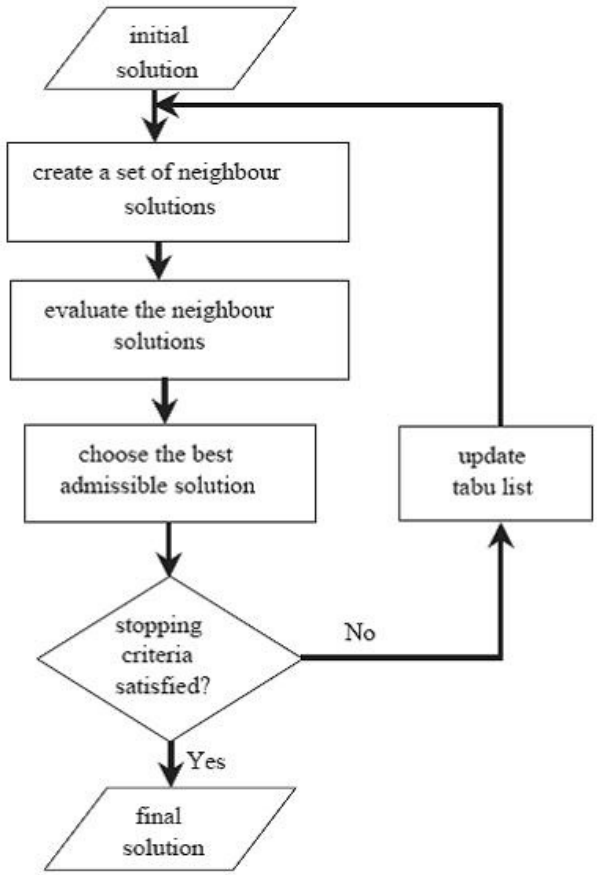


Fig. 2. Tabu Search schema.

called Tabu List for certain number of iterations named Tabu List size. This mechanism excludes moves which would bring the search back where it was at some previous iteration. In this simplest form, Tabu Search requires the following ingredients:

- Initial solution
- Neighborhood definition
- Move strategy
- Tabu List
- An integrated diversification scheme
- Stopping rules

Tabu Search is based on:

- The use of flexible memory structures allowing a complete exploration of the evaluation criterion and of the historical of research.
- A control mechanism based on the alternation between the restrict conditions (tabu restriction) and which release (aspiration criterion) the process of research.
- The intensification strategy using medium-term memory is used to Force search in the region of the best solutions found recently.
- The diversification strategy using the long-term memory, is used to guide Research in new regions.

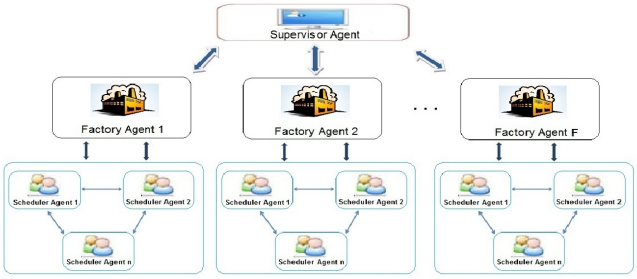


Fig. 3. Architecture of our model DTSMA

V. DISTRIBUTED TABU SEARCHES IN MULTI AGENT SYSTEM FOR DFJSP

We propose a multi agent model based on distributed tabu searches to solve the Distributed and Flexible Job shop Scheduling Problem (DFJSP) named "Decentralized Tabu Searches in Multi Agent system for Distributed and Flexible Job shop Scheduling Problem" (DTSMA) in order to minimize the maximum completion time or makespan (Cmax). The architecture of our model is composed of three classes of agents, a Supervisor Agent which is responsible for creating the initial solution and distributing the jobs to the different factories, *F* Factory Agents where *F* is the number of factories, which are responsible for scheduling in each factory and *n* Scheduler Agents, with *n* is the number of jobs in the problem, which are responsible of the optimization phase based on tabu search. See figure 3.

A. Supervisor Agent

This agent is considered as an interface between the user and the program and as supervisor of the all program sequences. It launches the program and creates the *F* Factory Agents with *F* is the number of factories and the *n* Scheduler Agents where *n* corresponds to the number of jobs, and then it generates the initial solution. It supplies the necessary information for every Factory Agent and for every Scheduler Agent to begin the global optimization phase; which is based on the Tabu Search, finally it posts the final result; the scheduling with the best Makespan. The Supervisor Agent has two types of knowledge. The static knowledge such as:

- The factories, the jobs and the operations to realize so its durations in the various resources.
- The initial solution created by the Supervisor Agent itself.
- The stopping criterion for the global optimization phase defined as nb-iteration.
- The necessary variables for the execution of the program.
- The found solutions after the phase of optimization.
- The best found solution after the phase of optimization.

And the dynamic knowledge as:

- The current initial solution.
- The all found solutions and its makespan (Cmax).

B. Factory Agent

This class of agents is responsible of the scheduling for each factory. Each agent receives its own solution and distributes the different jobs to different Scheduler Agents, then chooses the best makespan after each iteration of tabu search to it sends to Supervisor Agent. The static knowledge of the Factory Agent are cited as follows:

- The solution chosen by the Supervisor Agent.
- The tabu list size.

And the dynamic knowledge as:

- The jobs operations associated to the agent and its various durations.
- The all found solutions and its makespan.

C. Scheduler Agent

This class of agents aims to make the phase of optimization based on the Tabu Search. The static knowledge as:

- The number of iteration of the optimization phase.
- The other Scheduler Agents they belong to the same factory.

And the dynamic knowledge as:

- The jobs operations associated to the agent and its various durations.
- The current solution and its makespan.

D. Initial solution

The Supervisor Agent is responsible of this phase, the initial algorithm consists on affect first the different jobs on the all factories using a heuristic which consists in scheduling the current job in a each factory, then we choose the best scheduling. See algorithm 1.

Algorithm 1 Generate initial solution

```
1.begin-date (All factories) =0
2.end-date (All factories) = 0
3.Foreach i in jobs do
4.    Foreach f in factories do
5.        Foreach O_ij in operations do
6.            Choose a random machine r P_ijrf ≠ 0
7.                end-date (f) = begin-date (f)+P_ijrf
8.                begin-date (f) = end-date (f)
9.         EndFor
10.        end-date(f)=end-date(f)+travel-time(i,f)
11.    EndFor
12.EndFor
13.Choose the factory which gives a best scheduling
```

E. Optimization phase

1) The global optimization phase: This optimization phase is done by the work and the communication between the all Factory Agents and the all Scheduler Agents. So, after receiving the initial solution from the Supervisor Agent, each Factory Agent distributes this solution to each Scheduler Agent. Then each Scheduler Agent tries to make the tabu Search algorithm

then send the new found solution to each Scheduler Agent belongs to the same factory and to the Factory Agent. Finally, the Factory Agent chooses the best found solution and send it to the Supervisor Agent. This treatment is repeated until reach the number of iteration "nb-iteration". See algorithm 2.

Algorithm 2 Global Optimization algorithm

1. Input: *n*,*m*,*f*,nbop[],dure[][],travel-time[][],tabu-list-size, nb-iteration
2. Generate-initial-solution
3. While the stopping criterion not met {
4. Send (Supervisor Agent, Factory Agent, initial solution)
5. Distribute (Factory Agent, Scheduler Agent, initial solution)
6. Trigger Tabu Search (Scheduler Agent, initial solution)
7. Send (Scheduler Agent, Factory Agent, solutions)
8. Best-Cmax-Factory (Factory Agent, solutions)
9. Send (Factory Agent, Supervisor Agent, best-solution)
10. Best-Cmax (Supervisor Agent, Cmax)
11. } End while

2) Tabu Search algorithm: The Tabu Search (TS) metaheuristic was proposed by (Glover, 1986). This algorithm has been successfully used for solving different combinatorial optimization problems such as scheduling problems. It is a metaheuristic based on the local search. The idea of local search is to visit search space composed of all feasible solutions in order to reach the optimal solution.

Neighborhood function

We find in the literature different structure of neighborhood function, we can quote:

Swapping move: given a permutation of *n* jobs; *j* and $k \in \{1, \ldots, n\}$, two consecutive positions in π, the permutation that results of such a move is obtained by swapping the two jobs in position j and k. The size of the neighborhood is (*n* - 1). Experiments show that such a neighborhood yields local minima and requires high calculation time.

Exchange move: given a permutation π of *n* jobs; *j* and k $\{1, \ldots, n\}$, two positions in π, the permutation that results of such a move is obtained by placing the jobs in positions j and k. The size of the neighborhood is n (n -1)/2, these moves find good quality of schedules but the neighborhood exploration is in O (n^3 m) which leads to a high computational time.

Insertion move: given a permutation π of *n* jobs; *j* and $k \in \{1, \ldots, n\}$, two positions in π, the permutation that results of such move is obtained by exchanging the job from position *i* to position *j*. For all neighborhood types the positions *j* and *k* can be specified randomly or by applying some predefined criterion.

In our model, we choose the insertion move neighborhood structure. The move strategy is to each Scheduler Agent trying to change its operation in other positions (in inactivity intervals) to satisfy the various constraints such as the temporal constraints (precedence constraints) and the resource constraints. This treatment is repeated several times and in each iteration, each Scheduler Agent sends the new Cmax to Factory Agent.

TABLE II
THE PARAMETERS OF THE DTSMA

Parameter	Value
nb-iteration	1000
nb-diversification	5
Tabu List size	12

Evaluation of the current solution

Once the neighborhood step finished, the Factory Agent evaluates the neighborhoods of the current solution and chooses the best neighbor to be sent to Supervisor Agent.

Tabu List

The Tabu List is one of the mechanisms that attempted to avoid local optimality. A variety of elements can be stored in this list such as: makespans that was already obtained, a job and its position or a pair of jobs with their positions. In our model, we chose to apply a Tabu List defined as a finite list T with a fixed size experimentally determined which stores makespans of permutations already visited. If the number of the tabu list elements exceeds the tabu list size, we adopt the FIFO (First In First Out) strategy to delete the first arrival element from the tabu list.

3) Diversification phase: To guarantee better resolution of the Distributed and Flexible Job shop Scheduling Problem (DFJSP), we need to explore more search space. That's why diversification phase should be performed. When a better solution cannot be found after a certain number of iterations, the diversification phase is triggered. When the number of iterations is increased without a better solution after a certain number of iterations, it means that the best found solution was not replaced by one of its neighbors for a certain time, which is a sign that our method is probably trapped in a local optimum. In this situation, the Supervisor Agent chooses an initial solution from a set a available solutions found and chooses two jobs in different factories, then it swaps these two jobs and starts the optimization phase based on tabu search again.

VI. EXPERIMENTS

In this section, the performance of our proposed model is evaluated and tested. We have made series of experiments on benchmark instances proposed in the literature, namely instances of [20], [21] and [8]. These benchmarks represent problems that the number of job *N* varies in 10,15, 20 and 30, the number of resources *M* in [5 . .13], the number of operations by job [5 . . 10]. This gives problems such as the total number of operations in [50 . .300] and the number of factories *F* in [2 . . 4]. All tests were conducted on a I3-3110M 2*2.4 GHz with 4 GB. We use the Java object-oriented programming language with the Eclipse IDE, and for the development of our model DTSMA, we use the JADE platform. The parameters of our model are set in table 2. We tested our model on two groups of instances, the first group consists of instances of classical FJSP from the literature

TABLE III
HURINK INSTANCES WITH 2 FACTORIES

Instance	*N*	*M*	LB	IGA	GA-JS	DTSMA
La01	10	5	413	413	413	413*
La02	10	5	394	394	394	394*
La03	10	5	349	349	349	349*
La04	10	5	369	369	369	369*
La05	10	5	380	380	380	380*
La06	15	5	413	445	424	424
La07	15	5	376	412	398	398
La08	15	5	369	420	406	406
La09	15	5	382	469	447	463
La10	15	5	443	445	443	445
La11	20	5	413	570	548	553
La12	20	5	408	504	480	500
La13	20	5	382	542	533	551
La14	20	5	443	570	542	581
La15	20	5	378	584	562	597
La16	10	10	717	717	717	717*
La17	10	10	646	646	646	646*
La18	10	10	663	663	663	663*
La19	10	10	617	617	617	617*
La20	10	10	756	756	756	756*

adapted for the Distributed and Flexible Job shop Scheduling Problem, and the second group consists of instances of DFJSP.

A. Test on FJSP instances adapted for DFJSP

1) Hurink instances: We compared the results obtained by our model DTSMA on benchmark instances of [20] with 2 factories with the approach of [9] called an Improved Genetic Algorithm for the Distributed and Flexible Job-shop Scheduling problem (IGA) and the approach of [6] called a genetic algorithm embedded with a concise chromosome representation for distributed and flexible job-shop scheduling problems (GA-JS). The results shows that our model DTSMA reaches the optimum for 50% of instances. The tests demonstrate that our model DTSMA and IGA provide the same results in 55% of instances, while our model DTSMA surpasses IGA in 25% of instances. The results show also that our model DTSMA and GA-JS provide the same results in 65% of instances but GA-JS surpasses our model in 35 % of instances. the asterisk (*) indicates that optimal solution was found, see table 3.

We compared also the results obtained by our model DTSMA on benchmark instances of Hurink with 3 factories with IGA and GA-JS approaches. The results shows that our model reaches the optimum for 70% of instances. The tests demonstrate that our model DTSMA and IGA provide the same results in 70% of instances, while our model DTSMA surpasses IGA in 15% of instances. The results show also that our model DTSMA and GA-JS provide the same results in 70% of instances. the asterisk (*) indicates that optimal solution was found, see table 4.

We compared also the results obtained by our model DTSMA on benchmark instances of Hurink with 4 factories with IGA and GA-JS approaches. The tests demonstrate that our model reaches the optimum for 85% of instances. The results show that our model DTSMA, IGA and GA-JS provide the same

TABLE IV
HURINK INSTANCES WITH 3 FACTORIES

Instance	*N*	*M*	LB	IGA	GA-JS	DTSMA
La01	10	5	413	413	413	413*
La02	10	5	394	394	394	394*
La03	10	5	349	349	349	349*
La04	10	5	369	369	369	369*
La05	10	5	380	380	380	380*
La06	15	5	413	413	413	413*
La07	15	5	376	376	376	376*
La08	15	5	369	369	369	369*
La09	15	5	382	382	382	382*
La10	15	5	443	443	443	445
La11	20	5	413	425	413	417
La12	20	5	408	408	408	423
La13	20	5	382	419	398	401
La14	20	5	443	443	443	462
La15	20	5	378	451	420	434
La16	10	10	717	717	717	717*
La17	10	10	646	646	646	646*
La18	10	10	663	663	663	663*
La19	10	10	617	617	617	617*
La20	10	10	756	756	756	756*

TABLE V
HURINK INSTANCES WITH 4 FACTORIES

Instance	*N*	*M*	LB	IGA	GA-JS	DTSMA
La01	10	5	413	413	413	413*
La02	10	5	394	394	394	394*
La03	10	5	349	349	349	349*
La04	10	5	369	369	369	369*
La05	10	5	380	380	380	380*
La06	15	5	413	413	413	413*
La07	15	5	376	376	376	376*
La08	15	5	369	369	369	369*
La09	15	5	382	382	382	382*
La10	15	5	443	443	443	443*
La11	20	5	413	413	413	413*
La12	20	5	408	408	408	408*
La13	20	5	382	382	382	395
La14	20	5	443	443	443	451
La15	20	5	378	397	378	392
La16	10	10	717	717	717	717*
La17	10	10	646	646	646	646*
La18	10	10	663	663	663	663*
La19	10	10	617	617	617	617*
La20	10	10	756	756	756	756*

results also in 85% of instances. the asterisk (*) indicates that optimal solution was found, see table 5.

2) Brandimarte instances: We compared our model DTSMA on benchmark instances of [21] adapted for the DFJSP with the approach of [7] called Hybrid Genetic Algorithm (HGA) and the results show that the two models give the same results in two instances but HGA surpasses our model in the last instance. see table 6.

B. Test on DFJSP instances

We tested our model DTSMA in two instances of [8] and the results show that our model is better in term of makespan than the approach of [8] called a genetic algorithm with dominant

TABLE VI
BRANDIMARTE INSTANCES

Instance	N	M	F	HGA	DTSMA
MK1	20	6	2	51	51
MK2	20	6	2	39	39
MK3	30	8	2	210	264

TABLE VII
DFJSP INSTANCES

Instance	N	M	F	GADG	HGA	IGA	DTSMA
DFJSP 1	10	13	2	42	37	37	37
DFJSP 2	10	13	2	49	37	37	37

genes for DFJSP (GADG) in 100% of instances and gives the same results with the HGA and IGA approaches in all instances, see table 7.

VII. CONCLUSION AND PERSPECTIVES

In this paper, we study the Distributed and Flexible Job shop Scheduling Problem (DFJSP) and we propose, to solve this problem, a multi-agent model named "Decentralized Tabu Searches in Multi Agent system for Distributed and Flexible Job shop Scheduling Problem" (DTSMA) in order to minimize the maximum completion time or makespan. Our proposed model consists of a society agents and based on the tabu search metaheuristic for the optimization phase. Our model is composed of three classes of agents, a Supervisor Agent which is responsible for creating the initials solutions and of the distribution the jobs to the different factories, F Factory Agents where F is the number of factories, which are responible for scheduling in each factory and n Scheduler Agents, with n is the number of jobs in the problem, which are responsible of the optimization phase based on tabu search. The results obtained in our work now encourage us to continue to study the DFSP by using other metaheuristics such as Genetic Algorithms (GA), Chemical Reaction Optimization (CRO), Particle Swarm Optimization (PSO), etc.

REFERENCES

[1] E.L. Garey, D.S. Johnson and R. Sethi, "The Complexity of Flow-shop and Job-shop scheduling". Mathematics of Operations Research, vol. 1 pp. 117-129, 1976 .

[2] B. Marzouki and O. Belkahla Driss, "Multi Agent model based on Chemical Reaction Optimization for Flexible Job Shop Problem", International Conference on Computational Collective Intelligence, vol. 9329 no. 1, pp. 29-38, 2015.

[3] B. Marzouki, O. Belkahla Driss and K. Ghedira, "Multi Agent model based on Chemical Reaction Optimization with Greedy algorithm for Flexible Job shop Scheduling Problem", Procedia Computer Science, 2017, to appear.

[4] H.E. Nouri, O. Belkahla Driss and K. Ghedira, "Hybrid Metaheuristics within a Holonic Multiagent Model for the Flexible Job Shop Problem", Procedia Computer Science, no. 60, pp. 83-92, 2015.

[5] H.E. Nouri, O. Belkahla Driss and K. Ghedira, "Metaheuristics based on Clustering in a Holonic Multiagent Model for the Flexible Job Shop Problem", The Genetic and Evolutionary Computation Conference, pp. 997-1004, 2015.

[6] P.H. Lu , M.C. Wu , H. Tan , Y.H. Peng and C.F. Chen, "A genetic algorithm embedded with a concise chromosome representation for distributed and flexible job-shop scheduling problems", Journal of Intelligent Manufacturing, DOI 10.1007/s10845-015-1083-z, 2015.

[7] H.C. Chang and T.K. Liu, "Optimisation of distributed manufacturing flexible job shop scheduling by using hybrid genetic algorithms", Journal of Intelligent Manufacturing, pp. 1-14, 2015.

[8] F.T.S. Chan, S. H. Chung and P. L. Y. Chan, "Application of genetic algorithms with dominant genes in a distributed scheduling problem in flexible manufacturing systems", International Journal of Production Research, pp. 523-543, 2006.

[9] L. Giovanni and F.P. Pezzella, "An Improved Genetic Algorithm for the Distributed and Flexible Job-shop Scheduling problem", European Journal of Operational Research, vol. 200 pp. 395408, 2010.

[10] Z. Mohsen, "A heuristic algorithm for the distributed and flexible job-shop scheduling problem", The Journal of Supercomputing, DOI 10.1007/s11227-013-0986-8, 2013.

[11] T.K. Liu , Y.P. Chen and J.H. Chou, "Solving Distributed and Flexible Job-Shop Scheduling Problems for a Real-World Fastener Manufacturer", the journal for rapid open access publishing IEEE Access, 2015.

[12] F.T.S. Chan, S.H. Chung, L.Y. Chan, G. Finke and M.K. Tiwari, "Solving distributed FMS scheduling problems subject to maintenance: Genetic algorithms approach", Robotics and Computer-Integrated Manufacturing, vol. 22, pp. 493-504, 2006.

[13] S.H. Chung, H.C.W. Lau, G.T.S. Ho and W.H. Ip, "Optimization of system reliability in multi-factory production networks by maintenance approach", Expert Systems with Applications, vol. 36, pp. 10188-10196, 2009.

[14] M. Yadollahi and A.M. Rahmani, "Solving distributed flexible manufacturing systems scheduling problems subject to maintenance: Memetic algorithms approach", International Conference on Computer and Information Technology, DOI 10.1109/CIT.2009.113, 2009.

[15] I. Chaouch, O. Belkahla Driss and K. Ghedira, "Weaknesses of Ant System for The Distributed Job shop Scheduling Problem", International Conference on Enterprise Information Systems, Volume 1, pages 574-581, 2017.

[16] I. Chaouch, O. Belkahla Driss and K. Ghedira, "Elitist Ant System for The Distributed Job shop Scheduling Problem", International Conference on Industrial, Engineering, Other Applications of Applied Intelligent Systems, Proceedings, Part I, pages 112-117, 2017.

[17] I. Chaouch, O. Belkahla Driss and K. Ghedira, "A Survey of Optimization Techniques for Distributed Job Shop Scheduling Problems in Multi-factories", Cybernetics and Mathematics Applications in Intelligent Systems Proceedings of the 6th Computer Science On-line Conference, pages 369-378, 2017.

[18] H. Bargaoui, O. Belkahla Driss and K. Ghedira, "Minimizing Makespan in Multi-Factory Flow Shop Problem Using a Chemical Reaction Metaheuristic", IEEE Congress on Evolutionary Computation, DOI: 10.1109/CEC.2016.7744158, 2016.

[19] F. Glover, "Future paths for integer programming and links to artificial intelligence", Computers and Operations Research, vol. 13 pp. 533-49, 1986.

[20] J. Hurink, B. Jurisch and M. Thole, "Tabu search for the job-shop scheduling problem with multi-purpose machines", Operational Research Spektr vol. 15 no. 4 pp. 205-215, 1994.

[21] P. Brandimarte, "Routing and scheduling in a flexible job shop by tabu search", Annals of Operations Research vol. 4 no. 1-4 pp. 157-183, 1993.